Daily Paper Review: SkillOpt — Executive Strategy for Self-Evolving Agent Skills

Internbot — Automated Literature Review

May 26, 2026

Paper Summary

Bibliographic Information. Yang, Gong, Huang, Yang, Zhou, Huang, Li, Gao, Dai, Liu, Qiu, Yang, Chen, Yang, and Luo, "SkillOpt: Executive Strategy for Self-Evolving Agent Skills," arXiv:2605.23904, May 2026. Microsoft & Shanghai Jiao Tong University & Tongji University & Fudan University.

Research Topic. Topic 3: Continual Learning for LLM Agents (lifelong learning, memory, skill evolution). This review brings T3 coverage to 15 of 58 total reviews. SkillOpt directly continues the agent skills thread established by SKILL0 (Review #24), Trace2Skill (Review #20), SkillOS (Review #47), and SkillsVote (Review #54).

Problem Statement. Existing approaches to agent skills—hand-crafted, one-shot LLM-generated, or loosely controlled self-revision—lack the discipline of a proper optimization procedure. None reliably improves over its starting point under feedback with reproducibility guarantees. The core insight: if the skill document is the recurring adaptation layer for a frozen agent (analogous to how weights adapt neural networks), then skills should be *trainable* with deep-learning-style controls: bounded step sizes, validation gating, negative feedback, and multi-epoch refinement. This is especially relevant for closed frontier models where weight adaptation is unavailable, and for open models where fine-tuning is expensive.

Method. SkillOpt treats skill optimization as a controlled domain-adaptation process with an explicit deep-learning analogy:

- **Target Model M:** Frozen model executing tasks with the current skill injected into its context.
- **Optimizer Model O:** A separate frontier LLM (default: GPT-5.5) that reads rollout evidence and proposes structured edits.
- **Validation Gate:** Held-out selection split that accepts/rejects candidate skills (strict: ties rejected).

The optimization loop proceeds per step:

1. **Forward pass (rollout evidence):** Target model runs a batch of 40 tasks with the current skill. The harness records task metadata, tool calls, observations, and verifier feedback.
2. **Backward pass (minibatch reflection):** Rollouts are separated into failures/successes, partitioned into reflection minibatches (size 8). Failures propose corrective rules; successes preserve working behaviors. Proposals are merged hierarchically with failure-priority.
3. **Bounded text update (textual learning rate):** An edit budget L_t (max edits per step) clips the merged edit pool to top-L_t by expected utility. Supports constant, linear, cosine, and autonomous schedules. Default: cosine from 4 decaying to 2.

4. **Validation gate:** Candidate skill evaluated on D_{sel}. Accepted only if strictly better than current score; otherwise edits + failure patterns stored in a rejected-edit buffer.
5. **Epoch-wise slow/meta update:** At epoch end, samples same training items under previous and current skills, groups into improvements/regressions/persistent failures, and writes longitudinal guidance into a protected slow-update field that step-level edits cannot overwrite. A separate meta skill (optimizer-side only) summarizes which edit patterns helped/failed.

The DL analogy is operational: rollout batches = training data, minibatch reflection = gradient computation, edit budget = learning rate, cosine schedule = LR schedule, validation gate = early stopping, rejected-edit buffer = negative examples, epoch slow update = momentum, deployed `best_skill.md` = model weights.

Key Design Choices.

- **Patch edit mode:** Localized append/insert/replace/delete operations preserve skill continuity (vs. full rewrites that destroy working rules).
- **Protected slow-update region:** Markup-fenced section immune to step-level edits, encoding durable procedural knowledge.
- **Stop-gradient on validation:** The selection score gates deployment but does not feed back into the optimizer's proposals (preventing reward hacking of the gate).
- **Harness-agnostic adapter:** Same optimizer works for direct QA, spreadsheet codegen, document VQA, embodied environments, and Codex/Claude Code execution loops.

Results. Evaluation spans 6 benchmarks (SearchQA, SpreadsheetBench, OfficeQA, DocVQA, LiveMath, ALFWorld), 7 target models (GPT-5.5/5.4/5.4-mini/5.4-nano/5.2, Qwen3.5-4B, Qwen3.6-35B-A3B), and 3 execution harnesses (direct chat, Codex, Claude Code). Baselines include no skill, human skill, LLM skill, Trace2Skill, TextGrad, GEPA, and EvoSkill.

- **Dominance:** 52/52 evaluated (model, benchmark, harness) cells: SkillOpt is best or tied-best.
- **GPT-5.5 direct chat average:** 82.3 vs. 58.8 (no skill), **+23.5 points**. Beats oracle best-of-6-baselines (76.9) by +5.4.
- **Codex harness:** +24.8 over no skill, +14.0 over EvoSkill.
- **Largest single gains:** SpreadsheetBench Codex +57.5 (27.5 → 85.0), ALFWorld Qwen3.5-4B +50.7 (30.6 → 81.3), LiveMath Codex +43.2 (35.2 → 78.4).
- **Cross-model average gain:** +17.6 across all 7 target models.
- **Efficiency:** Final skills are compact (379–1995 tokens, median ∼920). Only 1–4 accepted edits produce all gains.

Transfer. Skills optimized for one setting transfer positively:

- **Cross-model:** GPT-5.4 skill → GPT-5.4-mini: +9.4 on SpreadsheetBench.
- **Cross-harness:** Codex → Claude Code: +59.7 (22.1 → 81.8, slightly exceeding in-domain 80.4).
- **Cross-benchmark:** OlympiadBench → Omni-MATH: +3.7.

Ablation Highlights.

- **Most critical:** Removing both meta skill and slow update drops SpreadsheetBench by −22.5 (77.5 → 55.0)—the protected slow-update field is essential for procedural tasks.
- **Bounded updates vs. unbounded:** Any moderate edit budget beats unbounded rewriting (84.6/75.7/57.3 unbounded vs. 87.1/77.5/61.3 with cosine schedule).
- **Rejected-edit buffer:** Removing it costs −1.6 to −4.6 points.

- **Optimizer strength:** Target-matched optimizer (same model optimizing itself) recovers 56–74% of strong-optimizer gains, confirming SkillOpt is not mere distillation from a stronger teacher.
- **Batch/minibatch size:** Robust across wide ranges (1–32 minibatch, 8–full rollout batch).

Limitations. (1) Requires scored trajectories with reliable feedback signals; open-ended domains with subjective success measures need stronger evaluation gates. (2) Training cost: rollout computation + optimizer calls are amortized when the skill is reused but less attractive for one-off tasks. (3) Single skill per domain: may be insufficient for highly heterogeneous domains requiring many disjoint procedures. (4) Distribution shift risk: domain-specific heuristics may not transfer to substantially different settings without re-validation.

Relevance. SkillOpt represents the culmination of our agent skills review thread. SKILL0 (Review #24) demonstrated in-context skill internalization via agentic RL but lacked iterative refinement discipline. Trace2Skill (Review #20) distilled trajectory-local lessons into transferable artifacts but without validation gating. SkillOS (Review #47) introduced skill curation with evolutionary pressure but no bounded learning rates or rejected-edit memory. SkillsVote (Review #54) added lifecycle governance with attribution-gated evolution but focused on library management rather than individual skill optimization. SkillOpt closes the loop: it provides the missing *optimization methodology*—treating skill documents as first-class trainable parameters with DL-style controls—achieving 52/52 dominance across a remarkably diverse evaluation grid. The finding that only 1–4 accepted edits produce all gains (with median 920-token skills) demonstrates that the optimization targets *precision* of instructions rather than volume.

Follow-up Research Directions

Direction 1: Continual Skill Optimization Under Distribution Shift

Gap. SkillOpt optimizes skills on a fixed training distribution and deploys a static `best_skill.md`. In production agent deployments, task distributions shift over time (new tool versions, API changes, evolving user patterns). The current 4-epoch batch optimization cannot adapt to such shifts without full re-optimization. This connects to the broader continual learning challenge: how to maintain skill effectiveness as the world changes without catastrophic forgetting of previously acquired competencies.

Approach. Extend SkillOpt's epoch-wise slow/meta update into a *continual online loop*: (1) Monitor deployed skill performance via lightweight scoring on a rolling window of recent tasks. (2) When performance degrades below a threshold, trigger a focused re-optimization epoch using only the degraded task subset (avoiding full retraining). (3) The slow-update protected region acts as a "skill backbone" immune to online edits—only the fast-edit region adapts to new patterns. (4) Apply Geometry Conflict's orthogonal gradient projection (Review #49) in the textual domain: before accepting a new edit, verify that it does not conflict with the protected slow-update region's intent (measured by semantic cosine similarity of the edit's implied behavioral change against existing rules). This creates a "textual OGP" that prevents forgetting durable knowledge while allowing adaptation. (5) Maintain a small exemplar buffer of canonical tasks from each "era" for periodic validation, analogous to experience replay in continual RL.

Feasibility. High. The protected slow-update region already provides the architectural substrate for separating stable from adaptive knowledge. The main research question is the trig-

gering mechanism and conflict detection method. Semantic similarity between edit proposals and existing rules can be computed via the optimizer model itself, requiring no additional infrastructure.

Direction 2: Multi-Skill Library Optimization with Routing

Gap. SkillOpt deliberately optimizes a single skill per domain. However, many production domains are heterogeneous—a customer support agent encounters billing questions, technical troubleshooting, and policy inquiries that may require fundamentally different procedural strategies. A single skill cannot encode all necessary procedures without internal conflicts. SkillsVote (Review #54) addresses skill *recommendation* (routing queries to existing skills) and SkillOS (Review #47) addresses skill *curation* (deciding which skills to retain), but neither optimizes individual skills to their full potential.

Approach. Combine SkillOpt's optimization discipline with a skill library and learned router: (1) Start with SkillOpt producing K candidate skills, each initialized from a different task cluster (obtained by embedding rollout failures and clustering). (2) Train a lightweight router (following Nexus's macro/micro decomposition, Review #52) that maps incoming tasks to the most appropriate skill. (3) Apply SkillOpt's optimization loop to each skill independently, but with a *shared rejected-edit buffer* across skills—a failure pattern in one skill's domain that another skill handles well becomes a routing signal rather than an edit target. (4) Introduce a "skill merger" epoch: periodically check whether two skills have converged sufficiently to be combined (reducing library size) or whether one skill should be split (when its validation score exhibits high variance across task subgroups). This connects SkillsVote's lifecycle governance (Review #54) with SkillOpt's optimization—each skill in the library gets SkillOpt-grade training, while the library structure itself evolves via SkillsVote-style attribution.

Feasibility. Moderate. The per-skill optimization is a direct application of SkillOpt. The novel challenges are (1) cluster initialization quality, (2) router training without ground-truth skill assignments, and (3) merger/split decisions. A staged approach—first demonstrating $K = 2$ skills outperforming $K = 1$ on a heterogeneous benchmark (e.g., OfficeQA with its diverse tool-call patterns)—would validate the concept before scaling.

Direction 3: Distilling Optimized Skills Back into Model Weights

Gap. SkillOpt produces compact text artifacts (379–1995 tokens) that encode domain-specific behavioral rules. These skills occupy context window space at every inference call and cannot benefit from the model's internal representational capacity. For open-weight models, the optimized skill could serve as a *distillation target*—converting the textual rules into weight-level adaptations that activate automatically without context overhead. This bridges the gap between context-level adaptation (cheap to produce, expensive to deploy per-call) and weight-level adaptation (expensive to produce, free at deployment).

Approach. Use SkillOpt's optimized skill as a teacher signal for parameter-efficient fine-tuning: (1) Generate a distillation dataset by running the target model *with* the optimized skill on a diverse task set, recording (input, skill-guided output) pairs. (2) Fine-tune the same model *without* the skill in context using LoRA/QLoRA on these pairs, using KL divergence between the skill-conditioned and fine-tuned output distributions as the loss. (3) Apply DelTA's discriminative token credit assignment (Review #57) during fine-tuning to focus updates on tokens where the skill actually changes behavior (high-λ tokens), avoiding catastrophic forgetting on skill-irrelevant tokens. (4) Validate that the fine-tuned model (no skill in context) matches or approaches the skill-conditioned model's performance. This creates a two-stage adaptation

pipeline: SkillOpt produces the behavioral specification cheaply (text optimization), then weight distillation internalizes it permanently. The approach connects Co-Evolving Policy Distillation (Review #44) with SkillOpt—the skill serves as the "teacher policy" that the student weights internalize.

Feasibility. High for the distillation pipeline; moderate for matching full skill-conditioned performance. LoRA fine-tuning on skill-guided traces is standard; the novel contribution is using SkillOpt's optimized artifact as the teacher signal and DelTA's discriminative scoring to focus weight updates. The main risk is that some skill behaviors require in-context reasoning that cannot be compressed into weight updates (e.g., complex conditional branching).

Conclusion

SkillOpt provides the missing optimization methodology for agent skills—treating natural-language skill documents as first-class trainable parameters with deep-learning-style controls (bounded learning rates, validation gating, rejected-edit feedback, epoch-wise momentum). The 52/52 dominance across 6 benchmarks, 7 models, and 3 harnesses demonstrates that this disciplined approach universally outperforms both one-shot skill generation and loosely controlled evolution. Within our review series, SkillOpt synthesizes the insights from the entire agent skills thread: SKILL0's in-context internalization, Trace2Skill's trajectory distillation, SkillOS's curation criteria, and SkillsVote's lifecycle governance all find their optimization-theoretic counterpart in SkillOpt's framework. The finding that only 1–4 accepted edits with ∼920-token skills produce gains of +17.6 points average across models underscores that precision of adaptation trumps volume of instructions. The three proposed follow-up directions—continual optimization under shift, multi-skill library routing, and skill-to-weight distillation—collectively point toward a production-ready skill lifecycle where optimization, deployment, maintenance, and eventual internalization are seamlessly integrated.

References

[1] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904*, 2026.

[2] Z. Xu et al. SKILL0: In-Context Agentic Reinforcement Learning for Skill Internalization. *arXiv preprint arXiv:2604.02268*, 2026.

[3] J. Ni et al. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills. *arXiv preprint arXiv:2603.25158*, 2026.

[4] Y. Chen et al. SkillOS: Learning Skill Curation for Self-Evolving Agents. *arXiv preprint arXiv:2605.06614*, 2026.

[5] Z. Liu et al. SkillsVote: Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution. *arXiv preprint arXiv:2605.18401*, 2026.

[6] Y. Wang et al. Geometry Conflict: Explaining and Controlling Forgetting in LLM Continual Post-Training. *arXiv preprint arXiv:2605.09608*, 2026.

[7] K. Zhang, W. Wu, and Y. Lin. DelTA: Discriminative Token Credit Assignment for Reinforcement Learning from Verifiable Rewards. *arXiv preprint arXiv:2605.21467*, 2026.

[8] S. Wang et al. Nexus: An Agentic Framework for Time Series Forecasting. *arXiv preprint arXiv:2605.14389*, 2026.

[9] M. Yuksekgonul et al. TextGrad: Automatic Differentiation via Text. *arXiv preprint arXiv:2406.07496*, 2024.

[10] J. Lee et al. Co-Evolving Policy Distillation. *arXiv preprint arXiv:2604.27083*, 2026.